Assignment 12.4

Name: Sunny Kolluri

HTNO: 2303A52444

Batch: 43

Task 1: Bubble Sort

Prompt:

Implement Bubble Sort in Python to sort student scores.
Add inline comments for comparisons, swaps, and passes.
Include early termination logic.
Explain best, average, and worst-case time complexity.
Show sample input and output.

Assignment 12.1.py > ...

```python
#Implement Bubble Sort in Python to sort student scores.
#Add inline comments for comparisons, swaps, and passes.
#Include early termination logic.
#Explain best, average, and worst-case time complexity.
#Show sample input and output.
# Bubble Sort implementation to sort student scores
def bubble_sort(scores):
    n = len(scores)
    # Early termination flag
    swapped = True
    # Pass counter
    passes = 0
    while swapped:
        swapped = False
        passes += 1
        # Inner loop for comparisons and swaps
        for i in range(n - 1):
            # Compare adjacent elements
            if scores[i] > scores[i + 1]:
                # Swap elements if they are in wrong order
                scores[i], scores[i + 1] = scores[i + 1], scores[i]
                swapped = True
    print(f"Number of passes: {passes}")
# Sample input: list of student scores
student_scores = [88, 75, 92, 85, 79]
print("Original scores:", student_scores)
# Sort the scores using bubble sort
bubble_sort(student_scores)
print("Sorted scores:", student_scores)
# Time Complexity Explanation:
# Best Case: O(n) - This occurs when the input list is already sorted. The algor
# Average Case: O(n^2) - This occurs when the input list is in random order. The
# Worst Case: O(n^2) - This occurs when the input list is sorted in reverse orde
```

TERMINAL

```
PS C:\Users\navya\OneDrive\Desktop\AI Assit> & "C:/Users/navya/OneDrive/Desktop/AI Assit/.venv/Scripts/Activate.ps1"
(.venv) PS C:\Users\navya\OneDrive\Desktop\AI Assit> & "C:/Users/navya/OneDrive/Desktop/AI Assit/.venv/Scripts/python.exe"
 Assit/Assignment 12.1.py"
Original scores: [88, 75, 92, 85, 79]
Number of passes: 4
Sorted scores: [75, 79, 85, 88, 92]
(.venv) PS C:\Users\navya\OneDrive\Desktop\AI Assit>
```

Observation:

Observation

- Bubble Sort compares adjacent elements and swaps them if needed.
- Early termination reduces unnecessary passes when the list is already sorted.

- For small lists (like student scores), it works fine.
- But for large datasets, it becomes slow because time complexity is $O(n^2)$.
- Best case happens when the list is already sorted → only 1 pass → $O(n)$.

Task 2: Nearly Sorted Data

Prompt:

Write Bubble Sort for nearly sorted roll numbers.
Suggest a better algorithm and implement Insertion Sort.
Explain why Insertion Sort is better for nearly sorted data.
Compare their behaviour with example input/output.

Assignment 12.1.py > ...

```python
# Bubble Sort implementation for nearly sorted roll numbers
def bubble_sort_nearly_sorted(roll_numbers):
    n = len(roll_numbers)
    swapped = True
    passes = 0
    while swapped:
        swapped = False
        passes += 1
        for i in range(n - 1):
            if roll_numbers[i] > roll_numbers[i + 1]:
                roll_numbers[i], roll_numbers[i + 1] = roll_numbers[i + 1], roll_numbers[i]
                swapped = True
    print(f"Bubble Sort Passes: {passes}")
# Insertion Sort implementation for nearly sorted roll numbers
def insertion_sort(roll_numbers):
    n = len(roll_numbers)
    for i in range(1, n):
        key = roll_numbers[i]
        j = i - 1
        # Move elements of roll_numbers[0..i-1], that are greater than key,
        # to one position ahead of their current position
        while j >= 0 and roll_numbers[j] > key:
            roll_numbers[j + 1] = roll_numbers[j]
            j -= 1
        roll_numbers[j + 1] = key
# Sample input: nearly sorted roll numbers
roll_numbers = [101, 102, 103, 105, 104, 106]
print("Original roll numbers:", roll_numbers)
# Sort using Bubble Sort
bubble_sort_nearly_sorted(roll_numbers.copy())
print("Bubble Sorted roll numbers:", roll_numbers)
# Sort using Insertion Sort
insertion_sort(roll_numbers.copy())
print("Insertion Sorted roll numbers:", roll_numbers)
# Explanation:
# Insertion Sort is better for nearly sorted data because it can quickly place elements in their correct position with fewer comparison
```

```
SyntaxError: '(' was never closed
(.venv) PS C:\Users\navya\OneDrive\Desktop\AI Assit> & "C:/Users/navya/OneDrive/Desktop/AI Assit/.venv/Scripts/python.exe" "c:/Users/navya/OneDr
 Assit/Assignment 12.1.py"
Original roll numbers: [101, 102, 103, 105, 104, 106]
Bubble Sort Passes: 2
Bubble Sorted roll numbers: [101, 102, 103, 105, 104, 106]
Insertion Sorted roll numbers: [101, 102, 103, 105, 104, 106]
(.venv) PS C:\Users\navya\OneDrive\Desktop\AI Assit>
```

Observation

- Bubble Sort still performs multiple comparisons even if the list is almost sorted.
- Insertion Sort performs fewer shifts when elements are already near their correct position.
- For nearly sorted roll numbers, Insertion Sort

Task 3: Searching Records

prompt

Implement Linear Search (unsorted data) and Binary Search (sorted data) in Python.
Add docstrings explaining parameters and return values.
Explain when Binary Search is applicable.
Compare time complexity and show sample input/output.

```python
#Implement Linear Search (unsorted data) and Binary Search (sorted data) in Python.
#Add docstrings explaining parameters and return values.
#Explain when Binary Search is applicable.
#Compare time complexity and show sample input/output.
# Linear Search implementation for unsorted data
def linear_search(arr, target):
    """
    Perform linear search on an unsorted array.

    Parameters:
    arr (list): The list of elements to search through.
    target: The element to search for.

    Returns:
    int: The index of the target element if found, otherwise -1.
    """
    for index in range(len(arr)):
        if arr[index] == target:
            return index
    return -1
```

```python
    return -1
# Binary Search implementation for sorted data
def binary_search(arr, target):
    """
    Perform binary search on a sorted array.

    Parameters:
    arr (list): The sorted list of elements to search through.
    target: The element to search for.

    Returns:
    int: The index of the target element if found, otherwise -1.
    """
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = left + (right - left) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
# Sample input for Linear Search
unsorted_data = [5, 3, 8, 1, 2]
target_linear = 3
print("Linear Search:")
result_linear = linear_search(unsorted_data, target_linear)
print(f"Target {target_linear} found at index: {result_linear}")

# Sample input for Binary Search
sorted_data = [1, 2, 3, 5, 8]
target_binary = 3
print("Binary Search:")
result_binary = binary_search(sorted_data, target_binary)
print(f"Target {target_binary} found at index: {result_binary}")
```

```python
# Explanation:
# Binary Search is applicable only when the data is sorted. It works by repeatedly dividing the search
# space in half, which allows it to find the target element in logarithmic time. The time complexity of Linear Search is O(n) b
```

```
(.venv) PS C:\Users\navya\OneDrive\Desktop\AI Assit> & "C:/Users/navya/OneDrive/Desktop/AI Assit/.venv/Scripts/python.exe" "c:/Users/navya/OneDrive/Desktop/A
 Assit/Assignment 12.1.py"
Linear Search:
Target 3 found at index: 1
Binary Search:
Target 3 found at index: 2
(.venv) PS C:\Users\navya\OneDrive\Desktop\AI Assit>
```

Observation

- Linear Search works on both sorted and unsorted data.
- Binary Search only works when the data is sorted.

- Linear Search checks each element → O(n).
- Binary Search divides the list repeatedly → O (log n).
- For large sorted datasets, Binary Search is much faster.

Task 4: Quick Sort vs Merge Sort

Prompt:

Complete recursive Quick Sort and Merge Sort in Python.
Add docstrings and explain recursion.
Compare best, average, and worst-case complexities.
Test on random, sorted, and reverse-sorted data.

```python
# Quick Sort implementation
def quick_sort(arr):
    """
    Sort an array using the Quick Sort algorithm.

    Parameters:
    arr (list): The list of elements to be sorted.

    Returns:
    list: A new sorted list.
    """
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quick_sort(left) + middle + quick_sort(right)
# Merge Sort implementation
def merge_sort(arr):
    """
    Sort an array using the Merge Sort algorithm.

    Parameters:
    arr (list): The list of elements to be sorted.

    Returns:
    list: A new sorted list.
    """
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left_half = merge_sort(arr[:mid])
    right_half = merge_sort(arr[mid:])
    return merge(left_half, right_half)
```

```python
def merge(left, right):
    """Merge two sorted lists into a single sorted list."""
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result
# Sample input for testing
random_data = [3, 1, 4, 1, 5, 9]
sorted_data = [1, 1, 3, 4, 5, 9]
reverse_sorted_data = [9, 5, 4, 3, 1, 1]
print("Quick Sort:")
print("Random Data:", quick_sort(random_data))
print("Sorted Data:", quick_sort(sorted_data))
print("Reverse Sorted Data:", quick_sort(reverse_sorted_data))
print("Merge Sort:")
print("Random Data:", merge_sort(random_data))
print("Sorted Data:", merge_sort(sorted_data))
print("Reverse Sorted Data:", merge_sort(reverse_sorted_data))
# Explanation:
# Both Quick Sort and Merge Sort are efficient sorting algorithms with an average and best-case time complexity of O(n log n).
# Quick Sort has a worst-case time complexity of O(n²) when the pivot is consistently the smallest or largest element.
# Merge Sort consistently has a time complexity of O(n log n) in all cases.
# Quick Sort is generally faster in practice due to better cache locality and lower constant factors, but Merge Sort is stable and guarantees O(n lo
```

```
Quick Sort:
Random Data: [1, 1, 3, 4, 5, 9]
Sorted Data: [1, 1, 3, 4, 5, 9]
Reverse Sorted Data: [1, 1, 3, 4, 5, 9]
Merge Sort:
Random Data: [1, 1, 3, 4, 5, 9]
Sorted Data: [1, 1, 3, 4, 5, 9]
Reverse Sorted Data: [1, 1, 3, 4, 5, 9]
(.venv) PS C:\Users\navya\OneDrive\Desktop\AI Assit>
```

Observation

- Both algorithms are much faster than Bubble and Insertion for large datasets.
- Quick Sort is usually faster in practical cases.
- Quick Sort worst case → $O(n^2)$ (if pivot is bad).
- Merge Sort always guarantees O (n log n).
- Merge Sort requires extra memory.
- Quick Sort uses less memory (in-place).

Task 5: Duplicate Detection

Prompt:

Write a brute-force duplicate detection algorithm using nested loops.
Analyse its time complexity.
Optimize using set or dictionary.
Compare both approaches and explain performance improvement.
Show sample input/output

```python
#Show sample input/output
def brute_force_duplicate_detection(arr):
    """Detect duplicates using nested loops."""
    duplicates = []
    for i in range(len(arr)):
        for j in range(i + 1, len(arr)):
            if arr[i] == arr[j] and arr[i] not in duplicates:
                duplicates.append(arr[i])
    return duplicates
def optimized_duplicate_detection(arr):
    """Detect duplicates using a set."""
    seen = set()
    duplicates = set()
    for num in arr:
        if num in seen:
            duplicates.add(num)
        else:
            seen.add(num)
    return list(duplicates)
# Sample input
sample_data = [1, 2, 3, 2, 4, 5, 3, 6]
print("Brute Force Duplicate Detection:")
print("Input:", sample_data)
print("Duplicates:", brute_force_duplicate_detection(sample_data))
print("Optimized Duplicate Detection:")
print("Input:", sample_data)
print("Duplicates:", optimized_duplicate_detection(sample_data))
```

Explanation:

The brute-force approach has a time complexity of $O(n^2)$ because it uses nested loops to compare each element with every other element.

The optimized approach using a set has a time complexity of $O(n)$ because it only iterates through the array once and uses constant-time lookups.

This results in a significant performance improvement for large datasets.

```
Brute Force Duplicate Detection:
Input: [1, 2, 3, 2, 4, 5, 3, 6]
Duplicates: [2, 3]
Optimized Duplicate Detection:
Input: [1, 2, 3, 2, 4, 5, 3, 6]
Duplicates: [2, 3]
(.venv) PS C:\Users\navya\OneDrive\Desktop\AI Assit>
```

Observation

- Brute-force method compares every element with every other element → $O(n^2)$.
- It becomes extremely slow for large datasets.
- Optimized method using set checks elements only once → $O(n)$.
- Set lookups are very fast (constant time).